Computer Graphics: Lecture 17

Spheres, Cube-maps, and Skyboxes

Welcome back!

Last time we learned the whole Phong model

We learned how to draw not only directional lights, but point lights as well. [what's the difference?]

We also learned about storing structs in WGSL, how it can help organize the software design of our shaders but how it also requires understanding *alignment*. [how does alignment work?]

This time

We're going to learn about generating spheres.

Really? That's all?

That's all…but it's a bit more challenging than you think.

We're also going to learn a new texturing technique: the cube map.
Finally: no more duplicating the UV coordinates on a cube.

Cube mapping isn't just useful for texture mapping cubes: it's great for spheres, too, and it even enables some cool features such as skyboxes and environment maps (shiny textures that reflect the environment).

Spheres

Okay, this is a geometry pop quiz. If you had to generate a sphere mesh *right now*, what would you do?

[Let's think about this as a class.]

Circles

Chances are, learning to generate a circle will help us understand how to generate a sphere.

So let's start with generating a circle.

Ultimately, our video card wants to draw triangles. It is technically possible to use fancy tesselation shaders to generate a circle that is visually perfect (i.e., at a given resolution it is impossible to tell it from an ideal circle).

But let's not worry about that now. Let's just generate a regular polygon with a lot of sides that we will interpret as a circle if it has enough sides.

Circles (2)

The most straightforward way to generate a circle is to first decide how many sides we want. Say, 100. Then, plot that many points in the right place.

```
const verts = []; // vertex data: x,y coordinates
const nPoints = 100;

for (let point = 0; point < nPoints; point++) {
    ...
}
```

So…[how do we know where a point goes? What are its coordinates?]

Circles (3)

These points are going to go around the circle. Over the course of this process, the points will complete a single turn.

That means, the first point will start at 0 turns, the second will then be at one hundredth of a turn, the third will be two hundredths, etc.

So, we can determine the number of turns like this:

```
for (let point = 0; point < nPoints; point++) {
    const turns = point / nPoints;
}
```

But that's not enough. How do we turn *turns* into XY coordinates?

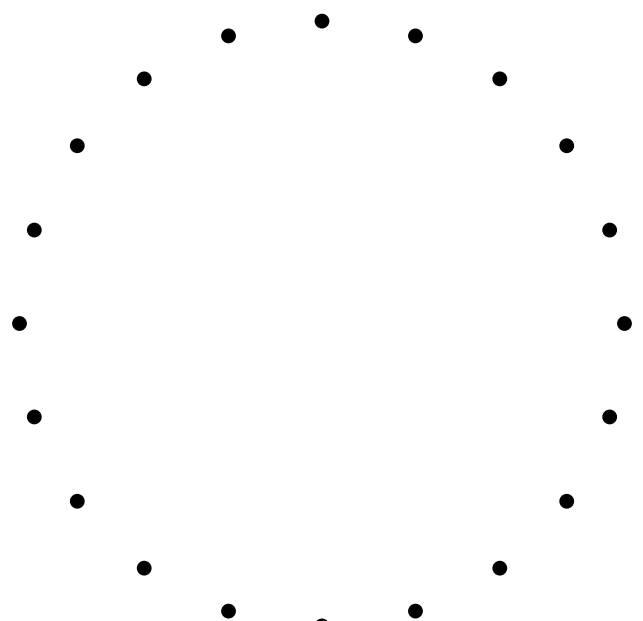
Circles (4)

First we need to convert them into angles in radians. Then we can use sine and cosine on the radians:

```
for (let point = 0; point < nPoints; point++) {
    const turns = point / nPoints;
    const rads = turns * TAU;
    const x = Math.cos(rads);
    const y = Math.sin(rads);
    verts.push(x, y);
}
```

Circles (5)

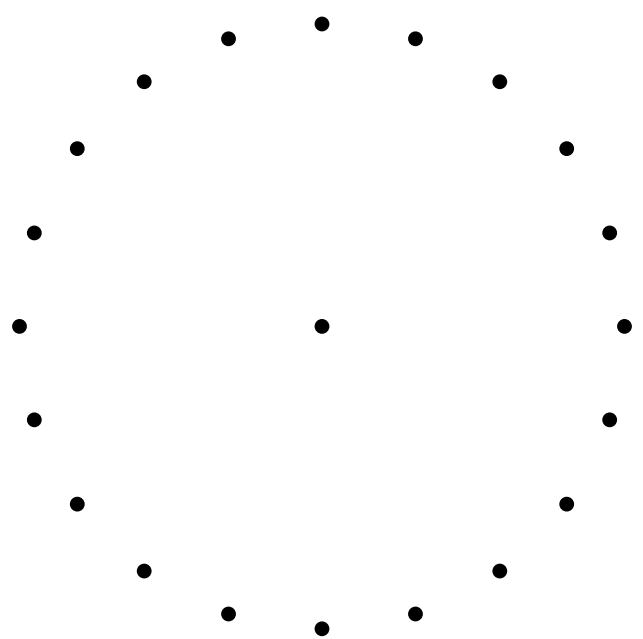
I coded the software renderer in my slides software to follow this algorithm using 20 vertices. It's pretty close to a circle.



Circles (6)

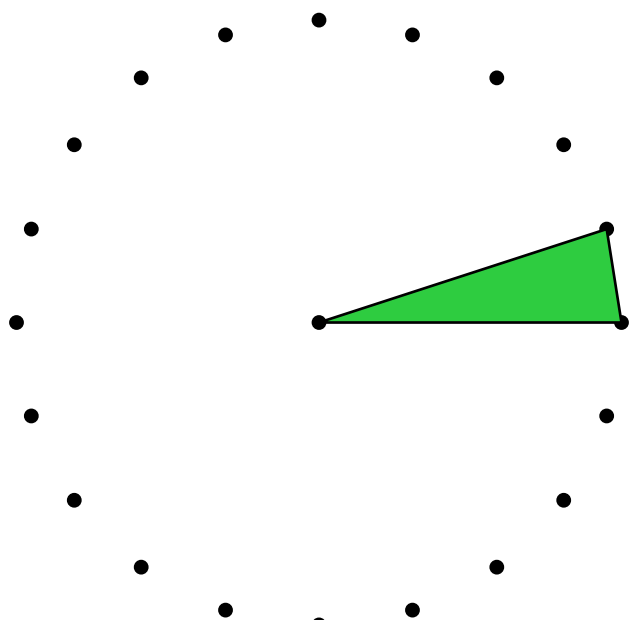
But a mesh isn't just a collection of points. It's also a collection of faces! (i.e., triangles). So how can we stitch our mesh together into triangles?

You might think we need to put a vertex in the center like this:



Circles (7)

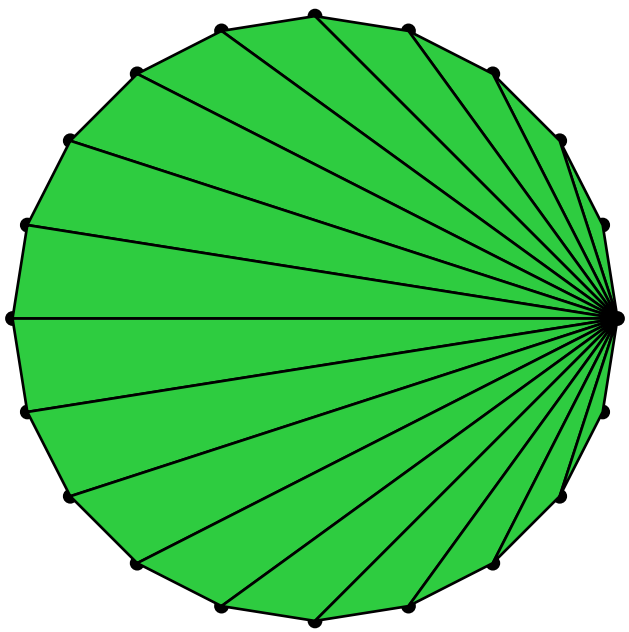
Then, we can stitch together a circle by creating triangles that pass through the center…



I.e., do this for all 20 slices.

Circles (8)

However, there is another way, where we just use one of the points on the side as the start of the triangle.



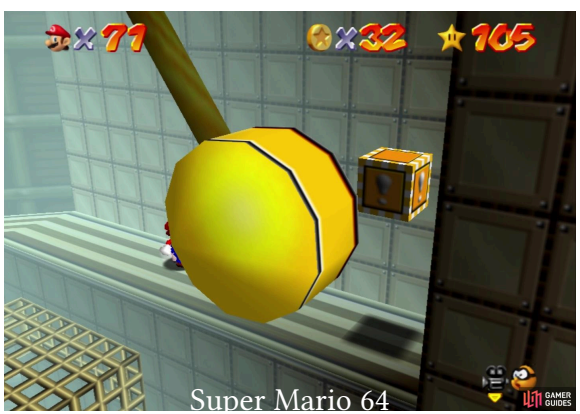
This saves a whole vertex 😮. Despite looking weird, it's just as good.

Triangle fans

There's a name for this kind of topology: a triangle fan.

It was built into legacy graphics APIs like OpenGL.

The first index in the strip would be the shared vertex, and then the other indices would be specified in the winding order. [0, 1, 2, 3, ...]



The pendulum's disc in Tick tock clock was a triangle fan.

Triangle fans (2)

Triangle fans were useful because you could generate any convex polygon from them. As long as the polygon was convex, you could use any point as the “center” point.

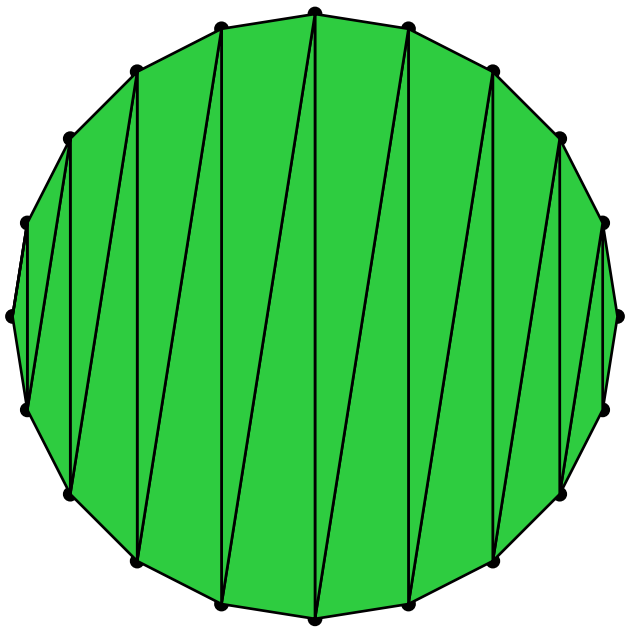
The screenshot before used the west most point in the disc.

However, WebGPU does not support triangle fans.

The reason is that it’s actually possible to use a triangle strip anywhere you would use a triangle fan, with the same number of indices.

[can anyone think of how to do it?]

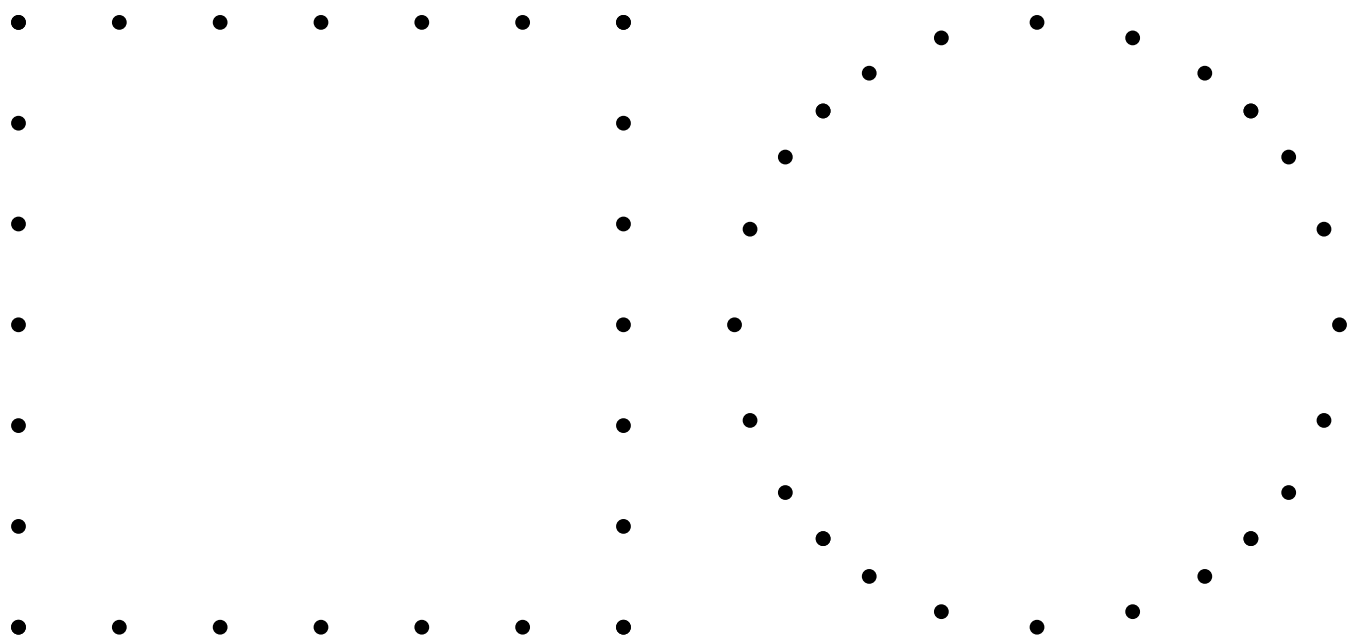
Triangle fans (3)



This saves a whole vertex and requires exactly as many indices as vertices! Just as good as a fan. [0, 1, 19, 2, 18, …]

Circles: more ways

There are actually several ways to generate circles. We can even start with a square and smoosh the points into a circle by normalizing them:



Before After

Circles: more ways

It works because a circle is a shape were every point on it has the same distance from the center. Assuming the center is 0, if we normalize all the points, the circle will have a radius of 1.

It's also not ideal. It ends up with the point distribution being uneven, as you can maybe see. We'd like the vertices to be spread more evenly.

However, it turns out that this is a great way of creating a sphere. It's called a cube sphere, and it's the *second* sphere algorithm we're going to learn today. Keep it in mind!

But first...

Questions?

Now spheres

Let's try to extend our algorithm for making circles.

Imagine that we made the equator of a large sphere that way.

Could we imagine how to stack more rings around the equator?

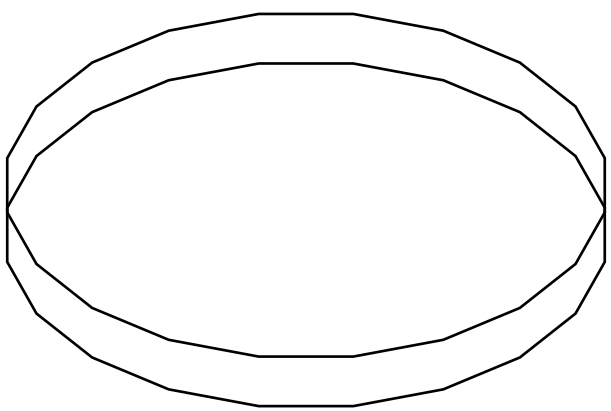
This kind of sphere is called a **UV-sphere**, because it's easy to compute the UV coordinates of its vertices.

[But how do we build one?]

UV-spheres

UV spheres are built out of rings, like the ones we generated earlier.

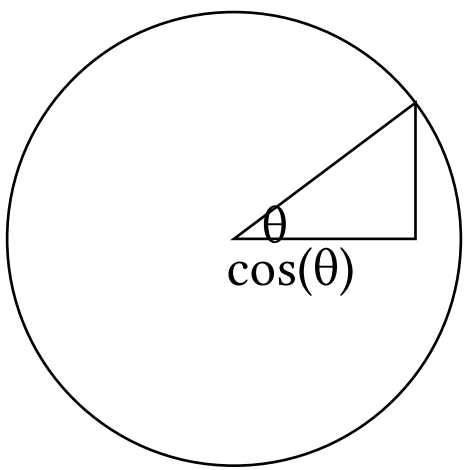
Only, we don't fill these rings into circles. We'll see how to tesselate them later.



Here are two rings. Assume the bottom is the equator. How do we shrink the top one to make it start to curve like the top of a sphere?

UV-spheres (2)

Let's imagine a vertical cross section. What we want is the cosine of the angle between two layers in the sphere:

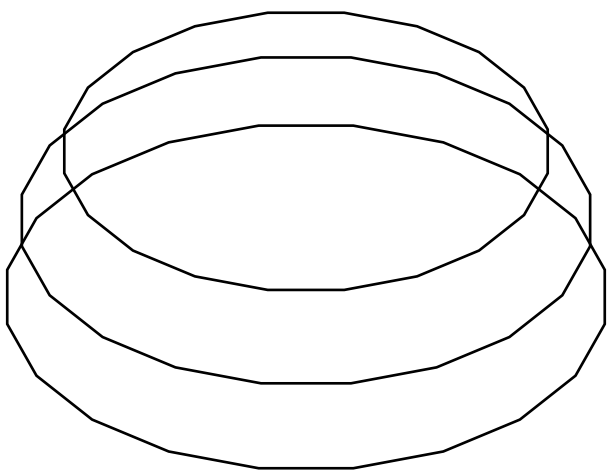


We can choose the angle of the height of the ring, or generate it. Either way the cosine of that angle is the radius of the ring.

UV-spheres (3)

And the y value is the sine of the angle.

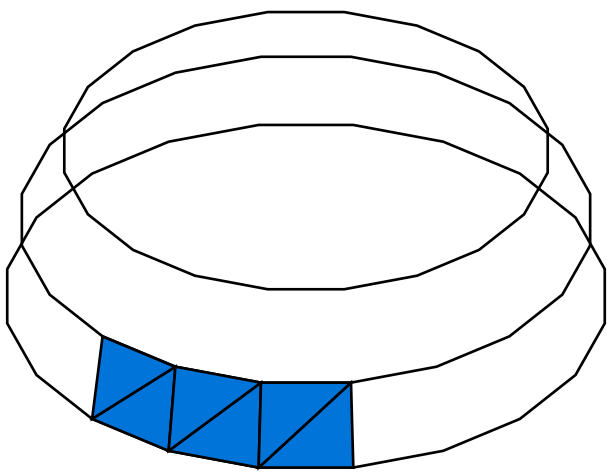
Let's stack 3 rings for good measure:



Hopefully we see the upper hemisphere start to form. But how do we form the triangles?

UV-spheres (4)

We pair adjacent rings together into strips. Then, we can draw a triangle strip all around the sphere. Here are some triangles: this pattern repeats:



We can draw the strip several ways. For example, if the points go clockwise: [0, pointsPerRing, 1, pointsPerRing + 1, 2, ...]

UV-spheres (5)

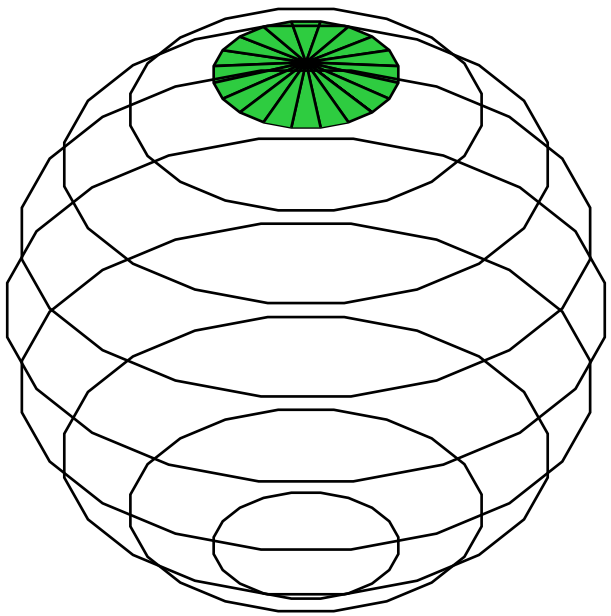
But what happens when we get to the top?

That's where it gets gnarly: you might think we'd put a single vertex at the top but we can't. Remember that these vertices need to have not only XYZ coordinates, but also UV.

If the top ring were connected to a single point, they'd all have to share the same UV coordinate, which would warp the texture.

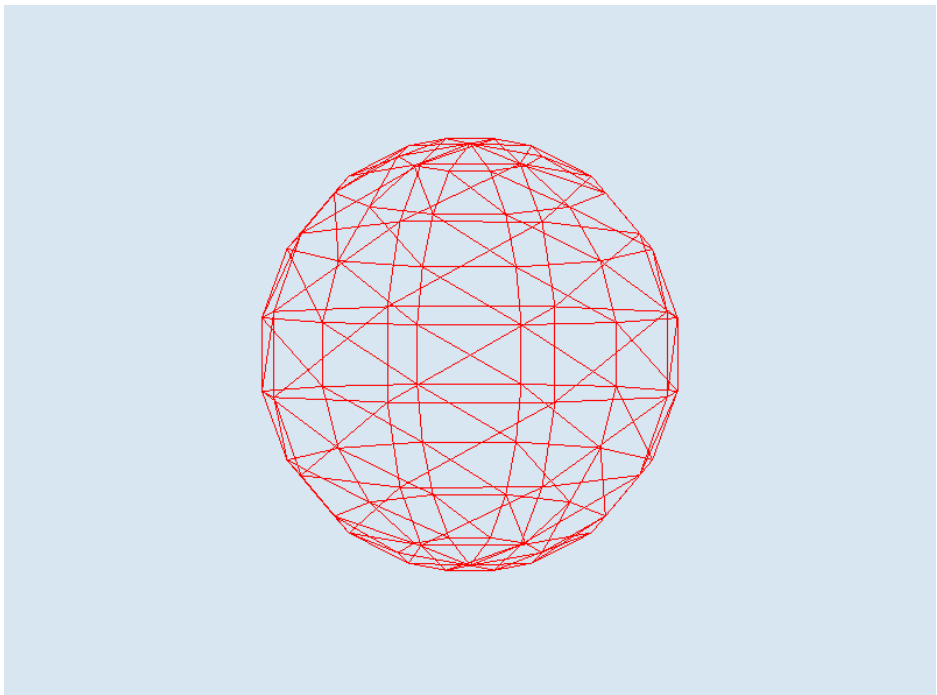
Instead we create an infinitely small ring at the very top. This allows each point in this ring to have a different UV coordinate, but also means that the top of the mesh is sealed (we say it's a watertight mesh)

UV-spheres (6)



Following the same triangle strip, half of our triangles are degenerate, because two adjacent points on the top ring are infinitely close together. You can special-case this if you want a slightly more optimized mesh.

Finished UV-sphere



A wireframe UV-sphere generated using the algorithm we've discussed.

See `sample14` for code.

UV-spheres (7)

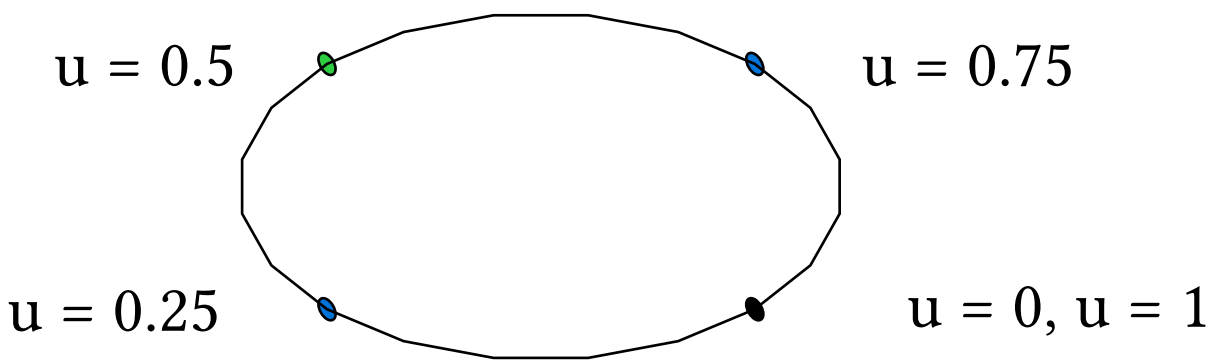
So far we've talked about how to generate rings out of vertices with known XYZ coordinates, and how to stitch those rings together into layers using indices. But what about UV coordinates?

The main point of UV spheres is that we want to wrap a single texture around them. So we want the U coordinate to increase as it goes around a latitude, and we want the V coordinate to increase as it goes toward the bottom pole.

So, [how can we compute UV coordinates?]

UV-spheres (8)

For a given ring, we can make the U coordinate of the first vertex 0. We want the last vertex to have a U coordinate of 1, and for it *to overlap the first vertex.*



In general, the `i`th point will have a U coordinate of `i / (numberOfPoints - 1)`, where `numberOfPoints` includes the last point that overlaps the first one.

UV-spheres (9)

But what about the V coordinate. We want the V coordinate to increase as the y value goes *down* (because remember, a V of 0 means the top of the texture).

[Any ideas?]

UV-spheres (10)

The simplest approach is, for ring number `i`, to just let `v = i / (num_rings - 1)`. This is assuming you're building the mesh from top to bottom.[1]

If you're going from bottom to top (so `i = numRings - 1` at the top ring), take the complement: `v = 1 - i / (num_rings - 1)`.

Alternatively, you can use an already-computed y component: `v = y * 0.5 + 0.5`. y normally ranges from −1 to 1, so this squeezes it into the range 0 to 1.

[1] some textures are distorted so that they are "squeezed" around the poles to counteract the fact that the y distance between rings is not constant. You can use the vertical angle of the ring and scale it to the range [0, 1] for these textures.

Questions?

UV-sphere problems

UV-spheres are a very simple way to draw a sphere with a texture painted over it. `sample14` shows how that can be used to draw a globe (the UV-sphere is on the left).

They aren't an ideal topology, however, and they are typically only used when we want to wrap a cylindrical texture.

[Can anyone think of a problem with them?]

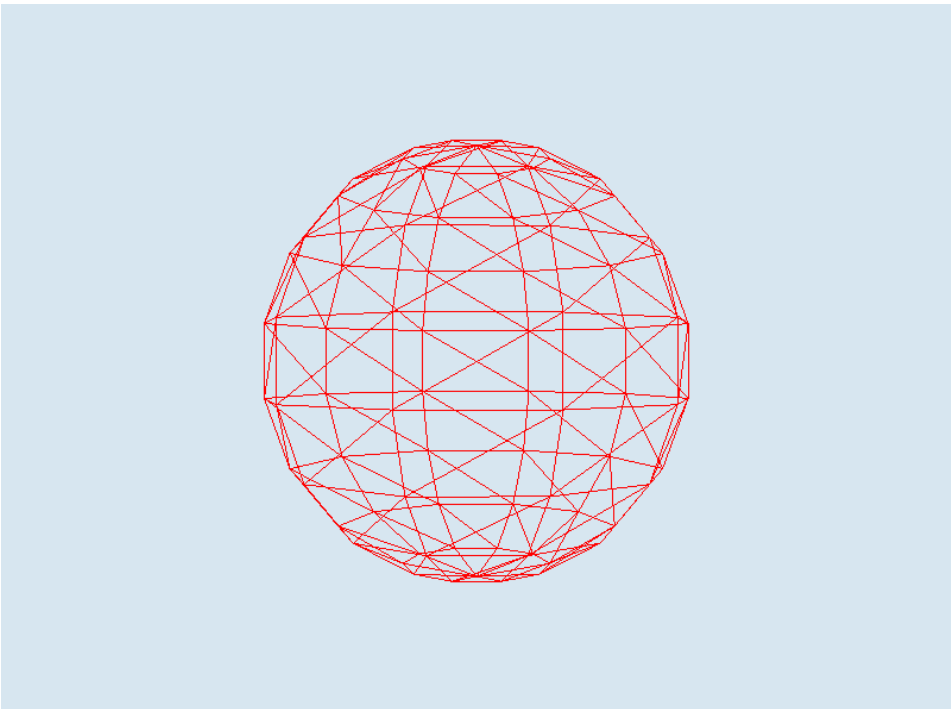
UV-sphere problems (2)

There are really two issues with UV-spheres:

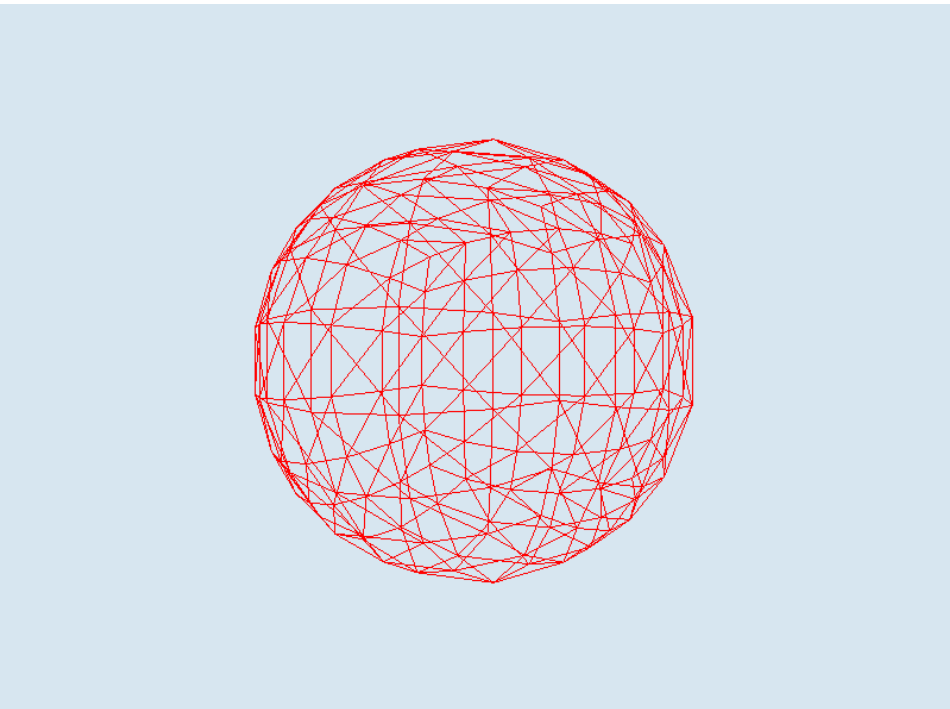
1. Poor point regularity: there are tons of points at the poles compared to the equator.
2. Texture distortion: as a result, the sphere has the lowest detail at the equator, where detail is often the most important!

To illustrate this, in the next slide, compare the wireframe of the UV-sphere to the wireframe of the next type of sphere we're going to learn…

UV-sphere vs Cube-sphere



A uv-sphere. Notice that the center faces are much larger than the ones near the top.



It's a little hard to tell, but this sphere's faces are much more evenly-sized.

Introducing cube-spheres

Let's learn how to make **cube-spheres** next. They have several advantages over UV-spheres:

1. They have better vertex distribution[1]
2. They can be more evenly textured

First, let's see how to build a cube-sphere. Then, we'll learn a cool new texturing technique that is ideal for them called cube-mapping.[2]

---

[1] although still not perfect: that property belongs to **icospheres**

[2] You can also use this technique on UV-spheres, but the texture will be sampled more unevenly, possibly causing minor distortion.

Building a cube sphere

Now it's time to build one.

Remember earlier in the lecture when we built a circle by starting with a square and then squishing it into a circle.

But this time it will be *good* instead of *bad*.

Wait…won't there still be distortion? Won't there be more points away from the faces of the cube?

Yes, but we're going to tolerate that, because it won't be as bad as the UV-sphere and it will texture better.

Building a cube sphere (2)

Let's start with the front face.

We want to, given a number of subdivisions, create a grid of points in a square shape. We can use a basic 2D for loop pair for this.

For each point, we want to normalize it so that we pull it into a sphere.

That's it. If we do that for a whole square, we will have one-sixth of a cube sphere.

Building a cube sphere (3)

```
const spanVerts = nFaceSubdivs + 1;
// front face, in the Z+ direction (right-handed)
for (let row = 0; row < spanVerts; row++) {
    for (let col = 0; col < spanVerts; col++) {
        const y = 2 * row / nFaceSubdivs - 1;
        const x = 2 * col / nFaceSubdivs - 1;
        const l = Math.sqrt(x * x + y * y + 1); // length
        // quiz: where does the 1 come from in the sqrt?
        // push the normalized coordinate:
        verts.push(x / l, y / l, 1 / l);
    }
}
```

Building a cube sphere (4)

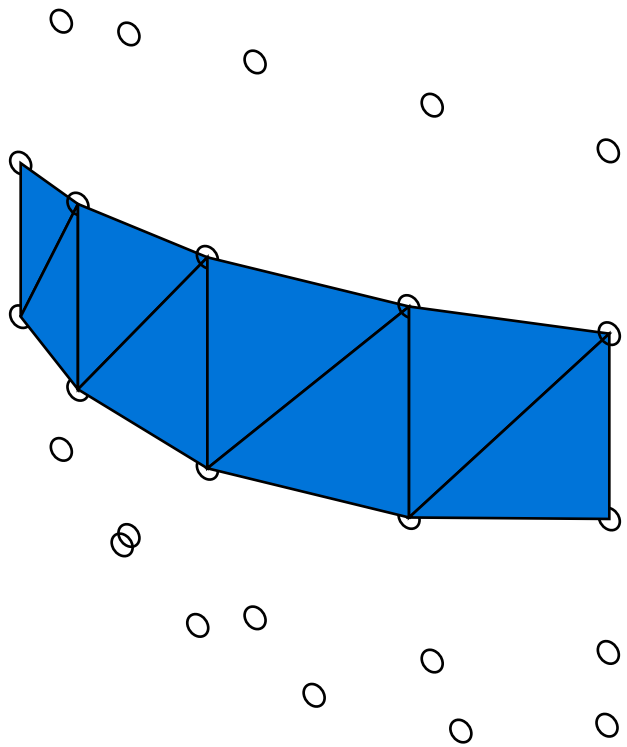
We do this for all six faces. The only wrinkle is making sure they all face *outward* (be mindful of winding).

But what about indices?

Believe it or not, that's actually easier than it was for the UV-sphere. We have a grid of points for each face. We just need to build a triangle strip.

The simplest way to do this is to build a "ribbon" going along each row…

Building a cube sphere (5)



Here is one such ribbon. I've rotated the image in 3D slightly so you can see how how it bends inward.

Building a cube sphere (6)

So, a triangle strip might look like this:

`[rowStart + vertsPerRow, rowStart, rowStart + vertsPerRow + 1, rowStart + 1, etc.`

And what happens at the end of the row?

…, `0xFFFFFFFF]`

That's the restart index. It tells the GPU to cut the strip.

So, we draw a perfectly normal grid of vertices, stitched into a triangle strip, except we also shrink the points until they are on the surface of a sphere. We do this for each face. That's a cube sphere.

Building a cube sphere...UVs?

So now you might wonder about UV coordinates.

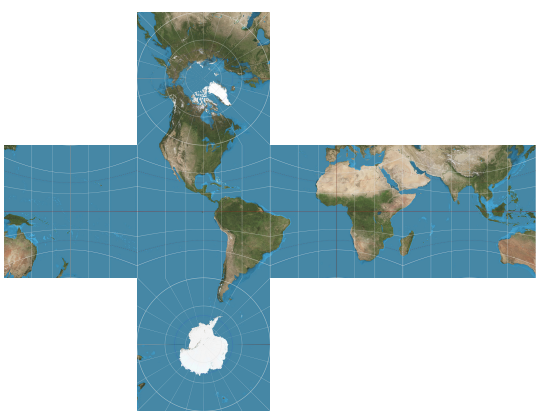
They're pretty straightforward if we want them to be: we can have each face sample a square in UV space. Let's say we wanted every face to have the same texture.

Then the top left corner of each face would have coordinates (0, 0), and the bottom right would have coordinates (1, 1).

But what if we *don't* want each face to have the same texture? Well, the techniques we've learned so far are somewhat suboptimal...

Cube sphere UVs (2)

One option is squeezing all 6 faces into a single texture. This works but it wastes a lot of memory and means we need a much bigger texture total. For example, I did this with the earth cube-sphere in `sample14`, and it required a 2048-by-1536 texture, but the sides were only 512-by-512, and only half of the texture is used:



Cube sphere UVs (3)

Another option is having 6 different textures. This is also unfortunate:

1. We could to treat our cube as 6 different objects. This would be very inefficient, because it requires 6 draw calls and bind group sets.
2. We could have 6 different textures in a bind group. This requires us to store an extra attribute, telling us which texture we want to lookup.

Luckily, there's a third option, that not only is perfectly space and texture efficient, but actually allows us to elemenate UV coordinates entirely!

It's called a **cube map**.

Cube maps

Cube maps are a special kind of texture which is indexed by a *position* instead of a *uv* coordinate.

Cube maps are built to assume they are being projected onto a cube. You use the *normal* of the mesh to look up the value.

Think of this as a 6-sided texture that wraps around your object, just like it's inside of a cube.

The normal is the vector pointing away from your object. The place where that vector intersects the cube is where we sample the texture.

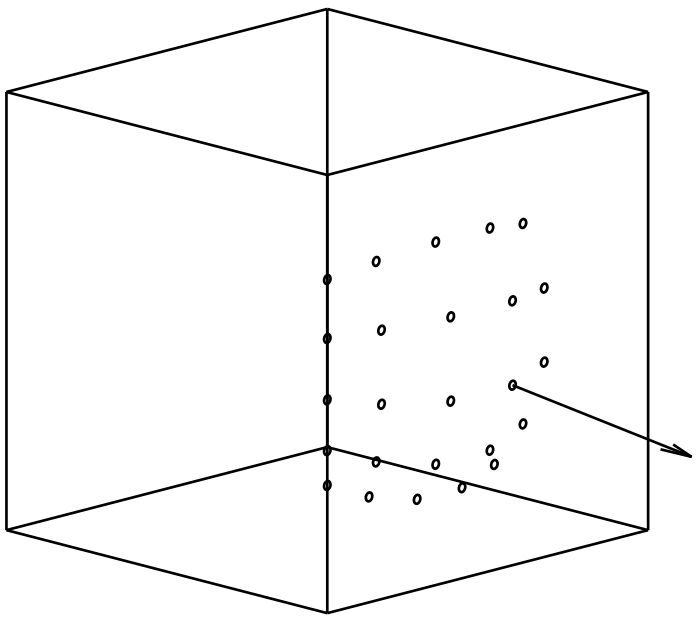
Cube maps (2)

What's interesting is: we can cube map any convex shape, and it doesn't require UV coordinates.

With a unit sphere, the XYZ coordinate is also the normal (i.e., the vector going from the center of the sphere to the surface has the same direction and length as the normal vector at the surface).

As a result, we can delete the UV coordinates and just use the cube-map.

Cube map illustration



One face of the cube-sphere, showing how at the given point, we sample the texture (the cube) at the point where the normal intersects it. The other faces would intersect the other sides of the cubemap.

The benefits of cube maps

Cube-maps are very versatile textures. They have a number of benefits:

- We can finally easily texture a cube without duplicating the vertices so that we can have different UV coordinates.
- We can use this feature for **skyboxes** (next slide).
- We can even use this feature to wrap the skybox *around* an object, making it so that the object appears to look shiny and reflect the sky. We can take this further and generate a cube map from the non-reflective environment and have it reflect things around it. This is called **environment mapping** (or **reflection mapping**).

Sky boxes

A **skybox** is a box that surrounds the camera to make it seem that there is a distant sky wrapping around the world.

Kenshi

Sky boxes (2)

Making a good skybox is challenging: you either need a special set of cameras, or you need a good texture artist who can make a large, seamless texture that can be viewed from any angle.

Luckily, *using* such a texture in a skybox is much easier.

We create a cube map with 6 different faces, and we create a cube. This cube will have the *opposite* winding order as a normal cube mesh, because we want to be *inside* it. It will wrap around the camera.

That is, the faces on the cube will be facing us from the inside.

Sky boxes (3)

Then, once we've made this texture and mesh, we make a shader. The shader will sample the skybox. It will use the orientation of the camera to determine what to sample (so as we look around, we will see different parts of the sky: we don't want a flat image).

We will still use the `textureSample` function, but now instead of giving it a UV coordinate to sample, we give it a normal.

How do we get the normal? When we draw the skybox mesh, it has 8 corners. We take those corners and normalize them. Those are our corner normals. Let's look at the sample to see it in action.

Questions?

Loading a cube-map

Loading a cube-map texture is just like loading a normal texture, except now we use the `depthOrArrayLayers` field:

```
const tex = device.createTexture({
    format: 'rgba8unorm-srgb',
    size: {
      width: faceW, height: faceH,
      depthOrArrayLayers: 6}, // this is the new part
    usage: GPUTextureUsage.TEXTURE_BINDING |
        GPUTextureUsage.COPY_DST |
        GPUTextureUsage.RENDER_ATTACHMENT,
    dimension: '2d',
});
```

`depthOrArrayLayers`

This field serves a few purposes:

- For 3D textures, it's the size of the 3rd dimension. Fully 3D textures are useful for things like smoke, fog, and fluids in general, as well as medical imaging.
- You can have arrays of 2D textures, which is useful for things like tilesets. This field is the length of the array.
- Lastly, for cube maps, it's always 6

Once we have the texture created, we copy over all 6 faces...

Loading the texture data

```
for (let face = 0; face < 6; face++) {
    device.queue.copyExternalImageToTexture(
        {source: datas[face]!, flipY: flips.includes(face)},
        {texture: tex, colorSpace: 'srgb', origin: [0, 0, face]},
        [dim, dim, 1]
    );
}
// these are large resources, let's free them properly.
await device.queue.onSubmittedWorkDone(); // wait until the copying is done
for (const data of datas) {
    data.close(); // close all 6 images to free the CPU memory
} // this *doesn't* delete the texture. the texture is in GPU memory.
```

(see the function `loadCubemap6images` in the sample to learn how `datas` is loaded efficiently.)

Loading the texture data

One last, important thing. The six textures in a cubemap have to come in a particular order:

1. +X (right)
2. -X (left)
3. +Y (top)
4. -Y (bottom)
5. +Z (front: right handed)
6. -Z (back: right handed)

That is, image with depth “0” is the right side of the cube. It’s what will get sampled if you sample points that have a positive dot product with (1, 0 0).

Creating a bind group layout

The default kind of texture is a basic 2D one. If we want to use a cubemap, we have to declare it as such in the bind group layout.

Here is what the entry in a bindgroup layout looks like for a cubemap:

```
{ // a single entry for one texture, see sample for rest.
    binding: 1,
    visibility: GPUShaderStage.FRAGMENT,
    texture: {
        // important!
        // this is how we "declare" to the pipeline that
        // we're expecting a cubemapped texture
        viewDimension: 'cube'
    }
}, // samp
```

Creating the bind group

Creating the bind group is similar, only, instead of passing the texture as the resource, we create a cubemapped view of it:

```
{
    binding: 1,
    resource: cubeMap.createView({
        // we're creating a cubemap "view" of our texture so
        // internally, it's ready to be sampled as a cubemap.
        dimension: 'cube',
        arrayLayerCount: 6,
    }),
},
```

A “view” is just a way of accessing a texture or buffer. The view also needs to know that it’s a cubemap.

The skybox shader

```
@vertex fn cube_vs(@location(0) pos: vec3f) -> VertexOutput {
    var vo: VertexOutput;
    vo.pos = vec4f(pos, 1.0f);
    // rotation only, no translation
    vo.world_pos = (model * vec4f(pos, 0.0)).xyz;
    return vo;
}

@fragment fn cube_fs(vo: VertexOutput) -> @location(0) vec4f {
    return textureSample(tex, samp, normalize(vo.world_pos));
}
```

We have a version of the position where we rotate it according to the "model" (a matrix that I use to orient the skybox). We make sure that only rotation is applied by setting the w component of pos to 0.

The skybox shader (2)

Once we apply the model transformation, the `world_pos` field represents the rotated vertex. We sample the cubemap using that rotated vertex. The effect is that we end up rotating the points that we're sampling every frame, so the skybox looks like it's spinning.

In general, we rotate the skybox according to the camera's view matrix. Then, we use its rotated corner vertices to sample the texture.

One more shader to cover: environment mapping…

The reflection on the sphere

This isn't the fully generalized way of doing environment mapping. It's a way that works in this specific orientation of the sphere. Normally we would need a normal matrix, but this sample was getting complicated enough as it was, and I added environment mapping last minute.

First, we use the rotation of the sphere *and* the rotation of the skybox to compute where the vertex is pointing in the sky cube:

```
vo.pos = proj * model * vec4f(pos, 1.0);
vo.model_pos = pos;
vo.sky_model_pos = (sky_model * model *
  vec4f(vo.model_pos, 0.0)).xyz;
```

The reflection on the sphere (2)

…except, that would give us the same orientation on the sphere as what the camera sees, which would not look realistic (we would see the same sky on the sphere as we see in the sky: it would not reflect the sky behind us).

So I did a janky hack and just inverted the z direction of the resulting vector. This works because in view space we're facing the sphere.

Normally, you would compute the normal in `view` space and use the reflection of that normal with the camera direction. I just took advantage of the fact that the camera direciton is fixed.

The reflection on the sphere (3)

We give the sphere a reflection by averaging the sample from the skybox texture with the sample from the world map texture:

```
let tex_color =
  textureSample(tex, samp, normalize(vo.model_pos));
var sky_sample = vo.sky_model_pos;
sky_sample.z *= -1;
sky_sample = normalize(sky_sample);
let sky_color = textureSample(sky, samp, sky_sample);
return (tex_color + sky_color) / 2.0; // blend colors
```

You can control the shininess by adding more of the sky_color and less of the texture color or vice versa, but I used the midpoint (even blend).

The passes

Two more new things:

1. We have multiple pipelines now. Because there are 3 different shaders, we need 3 different pipelines.
2. The skybox draws first and overwrites every pixel. Therefore, we no longer need to clear the screen. We disable the depth buffer, because the skybox is infinitely far away.

Let's take a look at the render pass descriptor now that we're using a skybox...

The render pass descriptor

```
const pass = encoder.beginRenderPass({
    colorAttachments: [{
        loadOp: 'load', // the skybox clears everything now
        storeOp: 'store',
        view: c.getCurrentTexture().createView({
            format: this.format,
        }),
        // clearValue: {r: .7, g: .8, b: .9, a: 1.0},
    }]
});
```

There are two changes here...

The render pass descriptor (2)

Formerly, we used `loadOp: 'clear'`, which would result in the screen being filled with the `clearValue`.

However, there is no point in clearing the screen if we have a skybox. The skybox will end up writing over every clear pixel anyway.

You might think “what’s wrong with clearing the screen anyway, it can’t be too slow”. Believe it or not, clearing the screen is rather slow. It requires writing to every pixel.

The speed at which video cards set pixels is called “fill rate”. Even modern cards are more limited in fill rate than you would think.

The render pass descriptor (3)

I don't want to overstate the case: clearing the screen is not a big deal.

...but it's nice to not do it if we don't have to.

You'll find that avoiding filling in large parts of the screen can help performance. For example, you might think it's no big deal to draw 1000 triangles, and that is normally true, but if they fill the screen and you draw them back to front, you are going to feel it.

Anyway, there was one more change in our render pass descriptor: we got rid of the depth attachment. We don't need it here because we're not drawing one sphere in front of the other one, and the skybox is always drawn first, so there's no depth ordering problems.

Conclusion

In conclusion, we learned about the following:

- How to generate and texture map a UV sphere
- How to generate a cube sphere
- How to load a cube texture
- How to use the cube texture in several ways:
  - To texture a cube
  - As a skybox
  - As an environment map

Next steps

Make sure you can build and understand the sample.

Then, look at project 5!

Questions?